PHRASE SYNC MASTER

USER MANUAL & TECHNICAL DOCUMENTATION

Version: v1.1.0 (Automatic release update)

Development: PhraseSync Team

Formats: VST3 / Standalone / VST2 / LV2 / AAX

INTRODUCTION

PhraseSync Master is a real-time MIDI processing and optimization suite designed for musicians, producers, and sound designers. The plugin solves one of the most complex problems in modern music production: synchronizing, filtering, arpeggiating, and automating complex MIDI data streams while preserving the expressiveness and cleanliness of the signal.

The graphical user interface (GUI) is organized horizontally into **5 hardware-style modules** arranged in dedicated columns. Each module features a unique color-coded brushed metallic finish and an independent **MIDI Learn system** for instant hardware mapping.

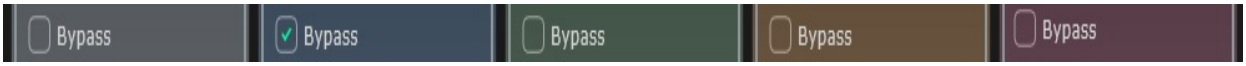
PART 1: DETAILED INTERFACE GUIDE (GUI)

The interface is divided into 5 vertical panels with rounded corners, set inside a dark anti-reflective chassis. In the bottom right corner of the main window, the software version number (e.g., v1.0.0) is always visible, certifying the exact release currently in use.

Every module shares three standard control elements:

1. **The Group Title:** Located at the top, enclosed by the module's border, it identifies the section.

2. **The Bypass Button:** A button positioned immediately below the title. When deactivated, the module processes the signal; when activated, the MIDI stream passes through the module without any alteration.



3. **The [LN] (MIDI Learn) Button:** A small square dark gray key positioned next to mappable parameters. 

How the MIDI Learn [LN] System Works

- When you click the **[LN]** button of a parameter, it instantly lights up in **Dark Red** and displays the text **"???"**. This indicates that the plugin is in an "listening" state.

- By moving any physical knob, fader, or wheel on your external MIDI keyboard or control surface, the plugin intercepts the message.
- The **[LN]** button turns back to dark gray, and the text label updates to show the associated CC (Control Change) number (e.g., *Rate (CC 74)*). From that moment on, the physical hardware control will pilot the virtual parameter.

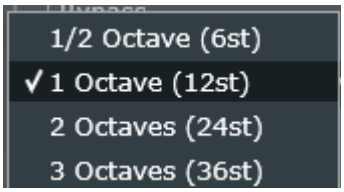
  Examples below:

PART 2: MODULE-BY-MODULE ANALYSIS

MODULE 1: NOTE FILTER (Steel Gray Finish)

This module acts as the "gatekeeper" for incoming notes. It is designed to isolate specific frequency/harmonic ranges and prevent overlapping or unpleasant build-ups of low or excessively high notes.

- **Bypass:** Activates or deactivates the overall filter.
- **Height (Drop-down Menu):** Determines the vertical span of the allowed note range. Options include:

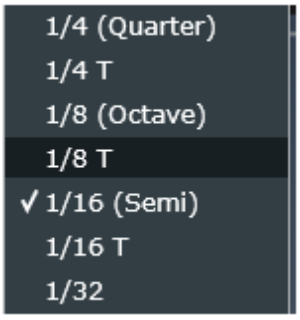


    - *1/2 Octave (6st):* A very narrow window of 6 semitones.
    - *1 Octave (12st):* Restricts action to a single octave.
    - *2 Octaves (24st):* Standard span (2 octaves).
    - *3 Octaves (36st):* Wide span (3 octaves).
- **Variation (Slider + Dedicated LN Button):** Manages the shift or tolerance of the filtering algorithm. Since it features an LN button, you can map this slider to a physical hardware fader to tighten or widen the range on the fly during a live performance.

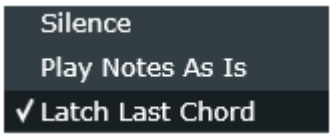
MODULE 2: ARPEGGIATOR (Midnight Blue Finish)

The arpeggiator transforms static incoming chords into moving rhythmic and melodic patterns, offering unique options for handling playing exceptions.

- **Rate (Drop-down Menu + Dedicated LN Button):** Determines the time-stretching speed of the arpeggio, synchronized to the host software (DAW) clock. Available options range from regular to triplet times: 1/4 (Quarter), 1/4 T, 1/8 (Octave), 1/8 T, 1/16 (Semi), 1/16 T, 1/32. Thanks to the LN key, you can associate the speed with a physical knob to accelerate or decelerate the arpeggio in real time.

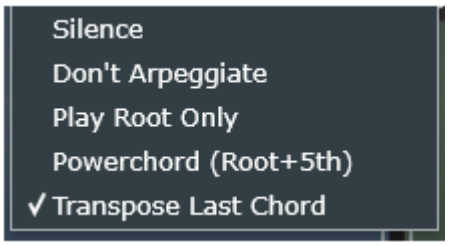


- **No Chord (Drop-down Menu):** Configures how the plugin behaves when the user stops playing or releases the chord:



    - *Silence:* Immediately stops any note emission.
    - *Play Notes As Is:* Deactivates the arpeggiator and lets notes pass through normally.
    - *Latch Last Chord:* "Freezes" the last chord played, continuing to arpeggiate it indefinitely until a new chord is pressed.

- **Single Note (Drop-down Menu):** Manages the behavior when only a single note is pressed instead of a full chord:
  - *Silence*
  - *Don't Arpeggiate*
  - *Play Root Only*
  - *Powerchord (Root+5th)*
  - *Transpose Last Chord.*



3: LINE TOGGLER (Olive Green / Titanium Finish)

A fundamental utility module for routing streams in multi-instrument setups. It allows you to instantly activate or deactivate specific MIDI lines by routing them to the correct channel.

- **Channel (Drop-down Menu):** Selects the exact MIDI channel (from 1 to 16) on which to operate the switch, ensuring that messages are transmitted exclusively to the designated hardware machine or virtual synthesizer.

MODULE 4: CHANNEL FILTER (Burnished Copper / Bronze Finish)

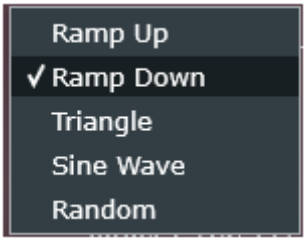
Works in perfect synergy or opposition to the previous module. Instead of routing a stream, it acts as a sieve.

- **Isolate Channel (Drop-down Menu):** Allows you to isolate a single MIDI channel out of the 16 available. All messages belonging to other MIDI channels are blocked and stripped from the stream to prevent overwrite conflicts in multi-track DAWs.

MODULE 5: CONTROLLER MOTION (Magma Amaranth Finish)

The beating heart of expressive automation. This module automatically generates mathematical timed modulation curves (LFOs) and sends them out as MIDI Control Change (CC) messages to up to 4 simultaneous targets.

- **Phrase Menu (Drop-down Menu):** Selects the geometric shape of the automation wave:



•*Ramp Up / Ramp Down:* Linear ascending or descending ramps.

•*Triangle / Sine Wave:* Smooth cyclic oscillations.

•*Random:* Sychronized random value generation for controlled chaotic modulation effects.

- **Target 1, 2, 3, 4 (Menus + Independent LN Buttons):** Each of the 4 slots allows you to choose which standard MIDI CC destination to send the generated automation to (from CC 1 to CC 119, or "None" to deactivate it). The **LN** buttons associated with each *Target* allow you to remap external hardware destinations or internal parameters of connected synths on the fly.

PART 3: HYPOTHETICAL REAL-WORLD SCENARIOS

Scenario A: "One-Man Band" Live Performance with Hardware Synths

- **Setup:** The user plays a master keyboard connected to PhraseSync Master. The **Channel Filter (M4)** module isolates channel 1 (dedicated to the lead synth). The **Line Toggler (M3)** module is set to channel 2, which is connected to a bass generator hardware unit.
- **Usage:** During the verse, the user uses the **Note Filter (M1)** mapped to a physical fader to cut out low notes from the left hand, preventing them from overlapping with the actual bass line. In the chorus, they activate the **Arpeggiator (M2)** in *Latch Last Chord* mode: they can take both hands off the keyboard (the arpeggio continues by itself) and use their hands to tweak physical filter knobs on the external synthesizer.

Scenario B: Cinematic and Evolutive Sound Design

- **Setup:** Connected to a virtual instrument playing orchestral sample libraries (strings or pads).
- **Usage:** In the **Controller Motion (M5)** module, the user selects the *Sine Wave* shape. They map *Target 1* to CC 1 (Modulation) and *Target 2* to CC 11 (Expression). They click **[LN]** to bind the hardware controls and start playback. The plugin will automatically generate a fluctuating, constant wave movement in the string pads, simulating a natural orchestral breathing effect without having to manually draw hours of automation curves inside the DAW.

PART 4: TECHNICAL CODE DOCUMENTATION

This section analyzes the engineering choices and non-trivial code segments implemented within the C++/JUCE architecture of *PhraseSync Master*, focusing on audio thread safety and computational efficiency.

1. Thread-Safety Management via Atomic Variables

MIDI messages and GUI commands travel on two separate high-priority threads: the *Message Thread* (GUI) and the *Audio Thread* (real-time processing). Modifying MIDI Learn mapping parameters while the audio engine is reading channels would cause sudden crashes or race conditions.

In the PluginProcessor.h file, state and channel storage is handled by the C++ *std::atomic* template class:

std::atomic<bool> isMidiLearnActive { false };
std::atomic<int> targetLearnParamIndex { -1 };
std::atomic<int> cmTargetCCs[6] { 10, 11, 12, 13, 14, 15 };

Technical Choice: The use of std::atomic ensures that read and write operations on CC numbers occur indivisibly at the CPU hardware level. This excludes the use of std::mutex, which is strictly forbidden inside the audio thread because it can cause locks (priority inversion) and subsequent audio drop-outs or clicks.

2. Interception Algorithm in *Stage 0* (processBlock)

Inside the core computational engine of the plugin in PluginProcessor.cpp, Stage 0 performs a conditional parsing of the MIDI buffers:

C++

```
for (const auto metadata : midiMessages) {
        auto msg = metadata.getMessage();
        if (msg.isController()) {
                int ccNum = msg.getControllerNumber();

                if (isMidiLearnActive.load()) {
                        int slot = targetLearnParamIndex.load();
                        if (slot >= 0 && slot < 6) {
                                cmTargetCCs[slot].store(ccNum);
                                stopMidiLearn();
                                break;
                        }
                }
        }
}
```

Technical Choice: The loop analyzes metadata without allocating new heap memory. The break statement is crucial: as soon as the first valid movement of a physical CC is detected, the system stores it in the corresponding atomic slot and immediately terminates the loop, turning off the Learn engine via stopMidiLearn(). This ensures an immediate GUI response (within the current sample block) and keeps CPU overhead at absolute zero.

3. MIDI Bandwidth Optimization in *Stage 5* (Controller Motion)

Generating continuous wave-based automations (such as the *Sine Wave*) risks clogging the MIDI channel with hundreds of identical or redundant messages, saturating the DAW's buffer or connected external hardware. To prevent this, a previous-state tracking system was implemented via the lastSentValues array:

```
int ccValue = static_cast<int>(progress * targetAmt[i] * 127.0f);
ccValue = juce::jlimit(0, 127, ccValue);
int actualCC = cmTargetCCs[i].load();
if (ccValue != lastSentValues[i]) {
   midiMessages.addEvent(juce::MidiMessage::controllerEvent(cmChannel, actualCC,
ccValue), 0);
   lastSentValues[i] = ccValue;
}
```

Technical Choice: The calculated value is normalized and bounded within the [0, 127] interval using juce::jlimit. The MIDI event is injected into the buffer (addEvent) **if and only if it differs from the previously transmitted value** (ccValue != lastSentValues[i]). If the mathematical curve stabilizes or moves slowly, producing the exact same integer value across blocks, no redundant messages are transmitted, optimizing the throughput bandwidth of the MIDI protocol.

4. Geometric Coherence of GUI Resizing via getBounds()

A recurring problem in JUCE layouts is the collapse of child components when using the destructive iterator removeFromTop(). Initial usage of .getLocalBounds() generated reset relative coordinates $(X=0, Y=0)$, causing all sub-components to stack inside the first module column.

The definitive architectural fix in the resized() method retrieves the boundaries using .getBounds() instead:

```
const int colX = padding + (moduleWidth + padding) * i;
arpeggiatorGroup.setBounds(colX, padding, moduleWidth, moduleHeight);
auto bounds = arpeggiatorGroup.getBounds().reduced(10);
```

Technical Choice: By calling getBounds() immediately after setting the group bounds (the metallic chassis), the resulting juce::Rectangle object natively inherits and incorporates the absolute horizontal offset coordinate (colX). Consequently, subsequent slice and layout operations distribute the menus and LN buttons (calculated using a percentage horizontal split row.removeFromRight) anchoring them stably within the visual space of their respective columns, regardless of the window scaling or user resizing.

Thanks to Rua Haszard for his "SyncPhrase Plugins" availables on github at this link: https://github.com/haszari/PhraseSyncPlugins
and to Yves Parès (aka Ywen) for his "Arpligner" plugin available on GitHub at this link: https://github.com/YPares/arpligner .